

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

OPERATING SYSTEM

WEEK 11 LECTURE 2

FILE : ON DISK FILE SYSTEM

WISH YOU ALL THE BEST

❑ LECTURE OUTLINE

- **General Characteristics of Ext2/Ext4**
- **Ext2/Ext4 Disk Data Structures**
- **Creating the Ext2/Ext4 Filesystem**

❑ **Ext2 – Extended File System**

- ★ **Second Extended Filesystem (Ext2) is native to Linux and is used on virtually every Linux system**
- ★ **Ext2 provides support for modern filesystem features with fast performance.**
- ★ **The Second Extended Filesystem (Ext2) introduced in 1994 is quite efficient and robust and is, together with its offspring Ext3, and now Ext4 the most widely used Linux filesystem**

SOC 3010

OPERATING SYSTEM

DISK FILE SYSTEM

CHARACTERISTICS

□ GENERAL FEATURES

1. FILE BLOCK SIZE

- * When creating an Ext2 filesystem, the system administrator may choose the optimal block size (from 1,024 to 4,096 bytes), depending on the expected average file length.**
- * For instance, a 1,024-block size is preferable when the average file length is smaller than a few thousand bytes because this leads to less internal fragmentation that is, less of a mismatch between the file length and the portion of the disk that stores it**
- * On the other hand, larger block sizes are usually preferable for files greater than a few thousand bytes because this leads to fewer disk transfers, thus reducing system overhead.**

□ GENERAL FEATURES

2. SELECTION OF INODES FOR EFFECTIVE USE OF DISK SPACE

- ★ When creating an Ext2 filesystem, the system administrator may choose how many inodes to allow for a partition of a given size, depending on the expected number of files to be stored on it. This maximizes the effectively usable disk space.

3. GROUPING OF DISK BLOCKS

- ★ The filesystem partitions disk blocks into groups. Each group includes data blocks and inodes stored in adjacent tracks. With this grouping, files stored in a single block group can be accessed with a lower average disk seek time.

□ GENERAL FEATURES

4. REDUCTION OF FILE FRAGMENTATION

- ★ The filesystem preallocates disk data blocks to regular files before they are actually used. Thus, when the file increases in size, several blocks are already reserved at physically adjacent positions, reducing file fragmentation.

5. FASTER SYMBOLIC LINKS

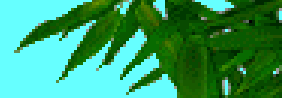
- ★ Fast symbolic links are supported. If the symbolic link represents a short pathname (at most 60 characters), it can be stored in the inode and can thus be translated without reading a data block.

❑ Second Extended Filesystem includes other features that make it both robust and flexible:

6. EFFICIENT FILE UPDATING TO MINIMIZE IMPACT OF SYSTEM CRASHES

- A careful implementation of file-updating that minimizes the impact of system crashes.**
- For instance, when creating a new hard link for a file, the counter of hard links in the disk inode is increased first, and the new name is added into the proper directory next.**
- In this way, if a hardware failure occurs after the inode update but before the directory can be changed, the directory is consistent, even if the inode's hard link counter is wrong.**
- Deleting the file does not lead to catastrophic results, although the file's data blocks cannot be automatically reclaimed.**
- If the reverse were done (changing the directory before updating the inode), the same hardware failure would produce a dangerous inconsistency: deleting the original hard link would remove its data blocks from disk, yet the new directory entry would refer to an inode that no longer exists.**
- If that inode number were used later for another file, writing into the stale directory entry would corrupt the new file**

GENERAL CHARACTERISTICS OF Ext2



7. SUPPORT FOR AUTOMATIC CONSISTENCY CHECKS ON THE FILESYSTEM STATUS

- Support for automatic consistency checks on the filesystem status at boot time.
- The checks are performed by the **e2fsck** external program, which may be activated not only after a system crash, but also after a predefined number of filesystem mounts (a counter is increased after each mount operation) or after a predefined amount of time has elapsed since the most recent check.

8. SUPPORT FOR IMMUTABLE FILES

- Support for immutable files (they cannot be modified, deleted, or renamed) and for append-only files (data can be added only to the end of them).

9. COMPATIBILITY WITH UNIX & BSD

- Compatibility with both the Unix System V Release 4 and the BSD semantics of the user group ID for a new file. In SVR4, the new file assumes the user group ID of the process that creates it; in BSD, the new file inherits the user group ID of the directory containing it. Ext2 includes a mount option that specifies which semantic to use.

GENERAL CHARACTERISTICS OF Ext2

8. SUPPORT FOR IMMUTABLE FILES

- Support for immutable files (they cannot be modified, deleted, or renamed) and for append-only files (data can be added only to the end of them).

```
sysadmin@ubuntu:~$ lsattr /etc/passwd
-----e-- /etc/passwd
sysadmin@ubuntu:~$
sysadmin@ubuntu:~$ sudo chattr +i /etc/passwd
sysadmin@ubuntu:~$
sysadmin@ubuntu:~$ lsattr /etc/passwd
----i-----e-- /etc/passwd
sysadmin@ubuntu:~$
sysadmin@ubuntu:~$ sudo rm /etc/passwd
rm: cannot remove '/etc/passwd': Operation not permitted
sysadmin@ubuntu:~$
```

Add immutable attribute

Check immutable attribute

chattr and lsattr commands are a part of e2fsprogs package which comes pre-installed on all modern Linux distributions.

Basic syntax of chattr is as follows.

```
$ chattr [-RVf] [operator][attribute(s)] files...
```

The operator can be '+' (which adds selected attributes to attribute list), '-' (which removes selected attributes from attribute list), or '=' (which forces selected attributes only).

Some of available attributes are the following.

a: can be opened in append mode only.

A: do not update atime (file access time).

c: automatically compressed when written to disk.

C: turn off copy-on-write.

i: set immutable.

s: securely deleted with automatic zeroing.

To make a file immutable, you can add "immutable" attribute to the file as follows. For example, to write-protect /etc/passwd file:

```
$ sudo chattr +i /etc/passwd
```

```
$ lsattr /etc/passwd
```

```
$ sudo chattr -R +i /etc
```

to make a whole directory (e.g., /etc) including all its content immutable at once recursively, use "-R" option:

```
$ sudo chattr -i /etc/passwd
```

To unset the immutable attribute

```
$ sudo chattr +a /var/log/syslog
```

turn a file into "append-only" mode



"The Ext3/ Ex4 Filesystem," a more recent filesystem that is fully compatible with Ext2 has been created, which offers the following additional features :

10. BLOCK FRAGMENTATION

- System administrators usually choose large block sizes for accessing disks, because computer applications often deal with large files. As a result, small files stored in large blocks waste a lot of disk space. This problem can be solved by allowing several files to be stored in different fragments of the same block.

11. HANDLING OF TRANSPARENTLY COMPRESSED AND ENCRYPTED FILES

- These new options, which must be specified when creating a file, allow users to transparently store compressed and/or encrypted versions of their files on disk.

12. Logical deletion

- An undelete option allows users to easily recover, if needed, the contents of a previously removed file.

13. Journaling

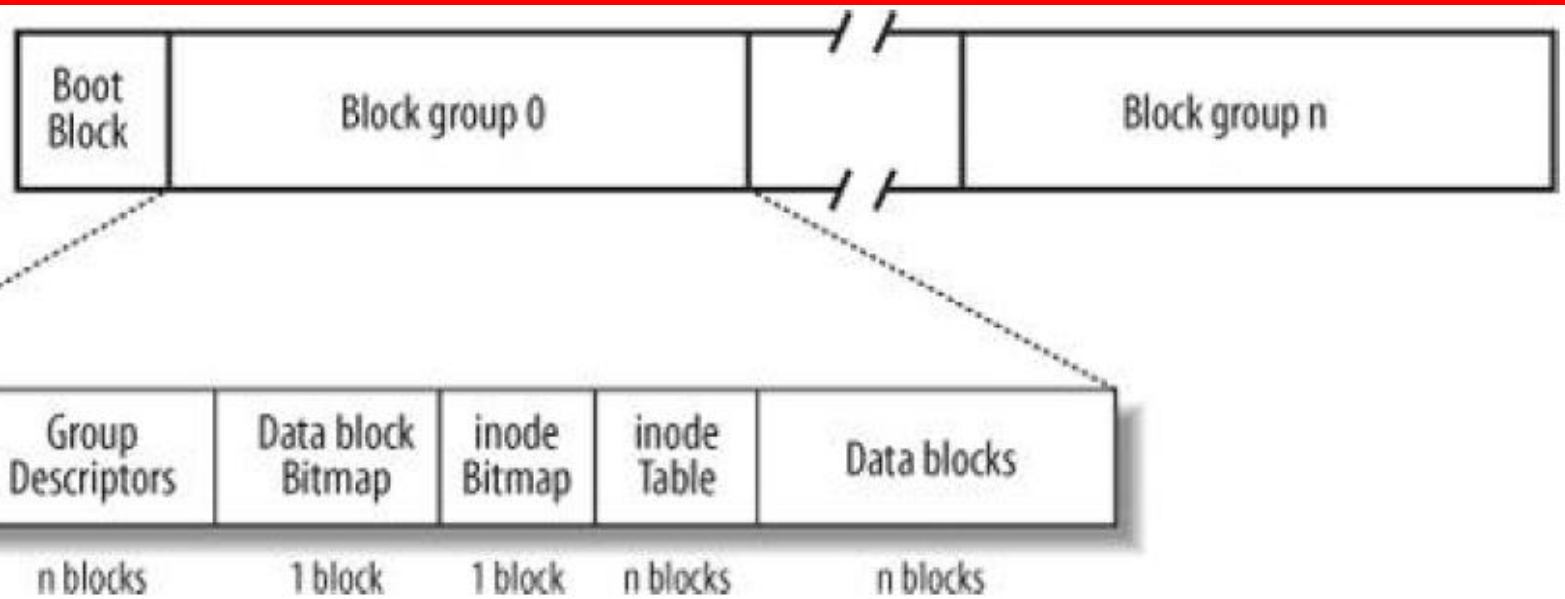
- Journaling avoids the time-consuming check that is automatically performed on a filesystem when it is abruptly unmounted for instance, as a consequence of a system crash.

SOC 3010

OPERATING SYSTEM

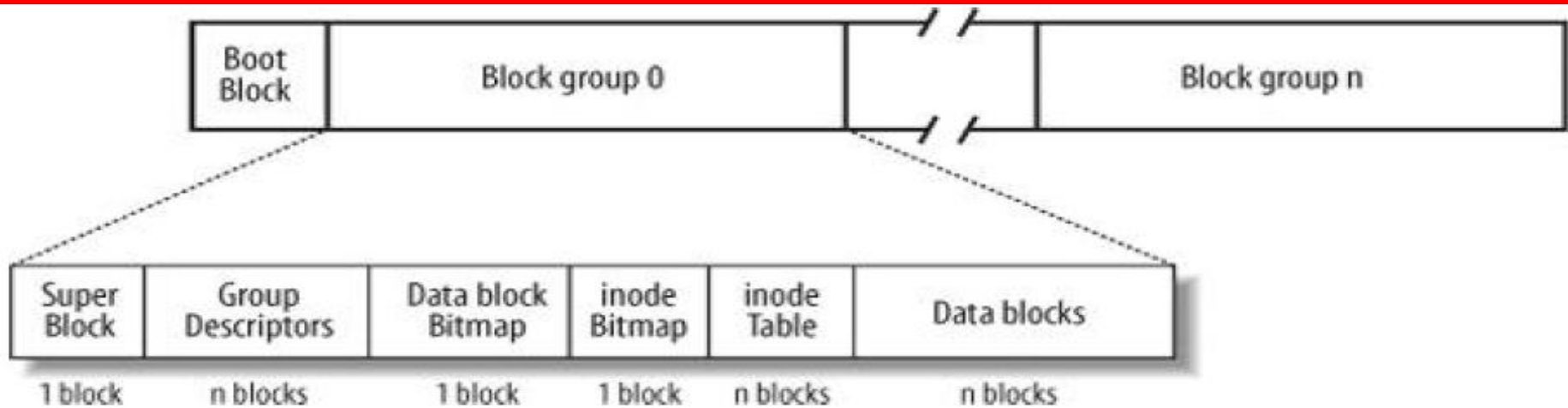
EXT2 DISK DATA STRUCTURES

Ext2 Disk Data Structures



- The first block in each Ext2 partition is never managed by the Ext2 filesystem, because it is reserved for the partition boot sector.
- The rest of the Ext2 partition is split into block groups , each of which has the layout shown in Figure.
- Some data structures must fit in exactly one block, while others may require more than one block.
- All the block groups in the filesystem have the same size and are stored sequentially, thus the kernel can derive the location of a block group in a disk simply from its integer index.

Ext2 FILE SYSTEM



- Block groups reduce file fragmentation, because the kernel tries to keep the data blocks belonging to a file in the same block group, if possible.
- Each block in a block group contains one of the following pieces of information:
 - A copy of the filesystem's superblock
 - A copy of the group of block group descriptors
 - A data block bitmap
 - An inode bitmap
 - A table of inodes
 - A chunk of data that belongs to a file; i.e., data blocks
- If a block does not contain any meaningful information, it is said to be free.

Ext2 FILE SYSTEM

- Both the superblock and the group descriptors are duplicated in each block group.
- Only the superblock and the group descriptors included in block group 0 are used by the kernel, while the remaining superblocks and group descriptors are left unchanged; in fact, the kernel doesn't even look at them.
- When the **e2fsck program** executes a consistency check on the filesystem status, it refers to the superblock and the group descriptors stored in block group 0, and then copies them into all other block groups.
- If data corruption occurs and the main superblock or the main group descriptors in block group 0 become invalid, the system administrator can instruct e2fsck to refer to the old copies of the superblock and the group descriptors stored in a block groups other than the first.
- Usually, the redundant copies store enough information to allow e2fsck to bring the Ext2 partition back to a consistent state.

Ext4 FILE SYSTEM LAYOUT

The layout of a standard block group is approximately as follows :

Group 0 Padding	ext4 Super Block	Group Descriptors	Reserve d GDT Blocks	Data Block Bitmap	inode Bitmap	inode Table	Data Blocks
1024 bytes	1 block	many blocks	many blocks	1 block	1 block	many blocks	many more blocks

- For the special case of block group 0, the first 1024 bytes are unused, to allow for the installation of x86 boot sectors and other oddities.
 - The superblock will start at offset 1024 bytes, whichever block that happens to be (usually 0).
 - However, if for some reason the block size = 1024, then block 0 is marked in use and the superblock goes in block 1.
 - For all other block groups, there is no padding.
- The ext4 driver primarily works with the superblock and the group descriptors that are found in block group 0.
 - Redundant copies of the superblock and group descriptors are written to some of the block groups across the disk in case the beginning of the disk gets trashed, though not all block groups necessarily host a redundant copy.
 - If the group does not have a redundant copy, the block group begins with the data block bitmap.
 - Note also that when the filesystem is freshly formatted, mkfs will allocate "reserve GDT block" space after the block group descriptors and before the start of the block bitmaps to allow for future expansion of the filesystem.
 - By default, a filesystem is allowed to increase in size by a factor of 1024x over the original filesystem size.

Ext4 FILE SYSTEM

- Ext4 divides a storage device into an array of logical blocks both to reduce bookkeeping overhead and to increase throughput by forcing larger transfer sizes.
- Generally, the block size will be 4KB (the same size as pages on x86 and the block layer's default block size), though the actual size is calculated as $2^{(10 + \text{sb.s_log_block_size})}$ bytes
- When `sb.s_log_block_size = 0`, Block size = $2^{10} = 1\text{KB}$
- When `sb.s_log_block_size = 2`, Block size = $2^{12} = 4\text{KB}$
- An ext4 file system is split into a series of block groups.
- To reduce performance difficulties due to fragmentation, the block allocator tries very hard to keep each file's blocks within the same group, thereby reducing seek times.
- The size of a block group is specified in `sb.s_blocks_per_group` blocks, though it can also be calculated as $8 * \text{block_size_in_bytes}$.
- With the default block size of 4KB, each block group will contain 32,768 Data blocks, for a length of 128MB.
- The number of block groups is the size of the device divided by the size of a block group.
- All fields in Ext4 are written to disk in little-endian order. HOWEVER, all fields in jbd2 (the journal) are written to disk in big-endian order.

Ext2 FILE SYSTEM

❑ How many block groups are there?

- Block groups depend both on the partition size and the block size.
- The main constraint is that the Data block bitmap, which is used to identify the Data blocks that are used and free inside a group, must be stored in a single block.
- Therefore, in each block group, there can be at most $8xb$ Data blocks, where b is the block size in bytes.
- Thus, the total number of block groups is roughly $s/(8xb)$, where s is the partition size in blocks.

Ext2 FILE SYSTEM

❑ How many block groups are there?

- For example,
- Consider a 32-GB Ext2 partition with a 1-KB block size. Determine the approximate number of Block Groups in the disk partition

Given: Disk partition size = 32 GB, Block size = 1 KB

- Block Size = 1KB = 1024 bytes = 1024×8 bits = 8192 bits
- Data block Bitmap size = 1 block = 1KB = 8192 bits
- Since each bit in the Data block bitmap corresponds to a Data block,
Total number of Data blocks in a Block Group = 8192 blocks = 8K
- In this case, each 1-KB block bitmap describes 8K data blocks
- Total amount of disk space required to store 8K data blocks
= 8K x size of each block = 8K x 1KB = 8MB.
- Size of a Blockgroup = 8K blocks = 8K x 1KB = 8MB
- Approximate Number of Block Groups in the given 32-GB partition
= $32\text{GB} / 8\text{MB} = 32 \times 1024 / 8 = 4096$ Block Groups
- Therefore, at most 4096 block groups are needed.
- Clearly, the smaller the block size, the larger the number of block groups.

Ext2 FILE SYSTEM

❑ How many block groups are there?

- For example,
- Consider a 32-GB Ext2 partition with a 4-KB block size. Determine the approximate number of Block Groups in the disk partition

Given: Disk partition size = 32 GB, Block size = 4 KB

- Block Size = 4KB = 4096 bytes = 4096x8 bits = 32768 bits
- Data block Bitmap size = 1 block = 4KB = 32768 bits
- Since each bit in the Data block bitmap corresponds to a Data block,
Total number of Data blocks in a Block Group = 32768 blocks = 32K
- In this case, each 4-KB block bitmap describes 32K data blocks
- Total amount of disk space required to store 32K data blocks
= 32K x size of each block = 32K x 4KB = 128 MB.
- Size of a Block Group = 32K blocks = 32K x 4KB = 128MB
- Approximate Number of Block Groups in the given 32-GB partition
= 32GB/128MB = 32x1024/128 = 256 Block Groups
- Therefore, at most 256 block groups are needed.
- Clearly, the larger the block size, the smaller the number of block groups.

Ext2 FILE SYSTEM

❑ How many block groups are there?

- For example,
- Consider a 64-GB Ext2 partition with a 4-KB block size. Determine the approximate number of Block Groups in the disk partition

Given: Disk partition size = 64 GB, Block size = 4 KB

- Block Size = 4KB = 4096 bytes = 4096x8 bits = 32768 bits
- Data block Bitmap size = 1 block = 4KB = 32768 bits
- Since each bit in the Data block bitmap corresponds to a Data block,
Total number of Data blocks in a Block Group = 32768 blocks = 32K
- In this case, each 4-KB block bitmap describes 32K data blocks
- Total amount of disk space required to store 32K data blocks
= 32K x size of each block = 32K x 4KB = 128 MB.
- Size of a Block Group = 32K blocks = 32K x 4KB = 128MB
- Approximate Number of Block Groups in the given 64-GB partition
= 64GB/128MB = 64x1024/128 = 512 Block Groups
- Therefore, at most 512 block groups are needed.
- Clearly, the larger the block size, the smaller the number of block groups.

Ext4 FILE SYSTEM

❑ How many block groups are there?

- Ext4 allocates storage space in units of "blocks".
- A block is a group of sectors between 1KB and 64KB, and the number of sectors must be an integral power of 2.
- Blocks are in turn grouped into larger units called block groups.
- Block size is specified at **mkfs** time and typically is 4KB.
- By default, a filesystem can contain 2^{32} blocks (for 32-bit mode);
- If the '64bit' feature is enabled, then a filesystem can have 2^{64} blocks

Ext4 FILE SYSTEM

File System Maximums		File System Maximums - 32-bit mode		
		32-bit mode		
Block size	1KB	2KB	4KB	64KB
Blocks	2^32	2^32	2^32	2^32
Inodes	2^32	2^32	2^32	2^32
File System Size	4TB	8TB	16TiB	256PB
Blocks Per Block Group	8,192	16,384	32,768	524,288
Inodes Per Block Group	8,192	16,384	32,768	524,288
Block Group Size	8MB	32MB	128MB	32GB
Blocks Per File, Extents	2^32	2^32	2^32	2^32
Blocks Per File, Block Maps	16,843,020	134,480,396	1,074,791,436	4,398,314,962,956 (really 2^32 due to field size limitations)
File Size, Extents	4TB	8TB	16TB	256TB
File Size, Block Maps	16GB	256GB	4TB	256TB

An extent is a contiguous area of storage reserved for a file in a [file system](#), represented as a range. A file can consist of zero or more extents; one [file fragment](#) requires one extent. The direct benefit is in storing each range compactly as two numbers, instead of canonically storing every block number in the range. Also, extent allocation results in less file fragmentation.

Ext4 FILE SYSTEM

File System Maximums - 64-bit mode

File System Maximums				
64-bit mode				
Block Size	1KB	2KB	4KB	64KB
Blocks	2^64	2^64	2^64	2^64
Inodes	2^32	2^32	2^32	2^32
File System Size	16ZB	32ZB	64ZB	1YB
Blocks Per Block Group	8,192	16,384	32,768	524,288
Inodes Per Block Group	8,192	16,384	32,768	524,288
Block Group Size	8MB	32MB	128MB	32GB
Blocks Per File, Extents	2^32	2^32	2^32	2^32
Blocks Per File, Block Maps	16,843,020	134,480,396	1,074,791,436	4,398,314,962,956 (really 2^32 due to field size limitations)
File Size, Extents	4TB	8TB	16TB	256TB
File Size, Block Maps	16GB	256GB	4TB	256TB

DISK FILE SYSTEM

□ Questions

- * Discuss general characteristics/features of Extended File System Ext2/Ex3/Ex4**
- * Explain how faster symbolic links are supported in ex2/ex4 file systems.**
- * Explain what is meant by consistency check of the filesystem. Indicate the command used in linux to perform consistency check of the filesystem.**
- * Explain what is meant by an immutable file. Write the command in linux to make a file immutable.**
- * Explain what is meant by append-only file. Write the command in linux to make a file append-only.**
- * Explain the purpose of the following commands :**
 - a) lsattr b) chattr c) ls -i d) extundelete**
- * State what happens when the following commands are executed :**
 - a) chattr +i /etc/passwd**
 - b) chattr +a file1**
 - c) lsattr /etc/passwd**
 - d) chattr -R +i dir1**
 - e) extundelete -Restore-all**
- * Explain the purpose of the following features in linux :**
 - a) Undelete b) Journaling**
- * With the help of block diagram, describe how EX2 disk partition is divided into block Groups and also draw the layout of a block group showing all the fields.**

DISK FILE SYSTEM

□ Questions

- * With the help of block diagram, describe how EX4 disk partition is divided into block Groups and also draw the layout of a block group showing all the fields.**
- * State the purpose of “Reserve GDT Block” in the Block group of Ext4 filesystem.**
- * Size of the Boot block in Ext4 partition is bytes.**
- * Distinguish between Ext2 and Ext4 filesystems.**
- * Draw the layout of a block group in Ex2/Ex4 partition showing all the fields and providing the purpose of each of the field and number of blocks required by each of the field.**
- * Describe the following :**
 - a) Super block b) Block group descriptor c) datablock bitmap**
 - d) inode bitmap e) inode f) inode table**
- * All fields in Ext4 are written to disk in byte order. However, all fields in jbd2 (the journal) are written to disk in byte order.**
- * State the reason to keep a copy of the superblock and a copy of the group of block group descriptors in each block group?**
- * Consider a 32-GB Ext2 partition with a 8-KB block size. Determine the approximate number of blockgroups in that partition.**
- * Consider a 512-GB Ext2 partition with a 4-KB block size. Determine the approximate number of blockgroups in that partition.**
- * Consider a 512-GB Ext2 partition with a 8-KB block size. Determine the approximate number of blockgroups in that partition.**

SOC 3010

OPERATING SYSTEM

BLOCK GROUP DATA STRUCTURES

SUPERBLOCK & BLOCKGROUP DESCRIPTORS

❑ Superblock

- The superblock records various information about the enclosing filesystem, such as block counts, inode counts, supported features, maintenance information, and more.
- In Ext4, If the **sparse_super** feature flag is set, redundant copies of the superblock and group descriptors are kept only in the groups whose group number is either 0 or a power of 3, 5, or 7.
- If the flag is not set, redundant copies are kept in all groups.

Ext2 FILE SYSTEM

❑ Superblock

- An Ext2 disk superblock is stored in an **ext2_super_block** structure, whose fields are listed in Table.
- The `__u8`, `__u16`, and `__u32` data types denote unsigned numbers of length 8, 16, and 32 bits respectively, while the `__s8`, `__s16`, `__s32` data types denote signed numbers of length 8, 16, and 32 bits.
- To explicitly specify the order in which the bytes of a word or double-word are stored on disk, the kernel also makes use of the `__le16`, `__le32`, `__be16`, and `__be32` data types; the former two types denote the little-endian ordering for words and double-words (the least significant byte is stored at the highest address), respectively, while the latter two types denote the big-endian ordering (the most significant byte is stored at the highest address).

Ext2 FILE SYSTEM

❑ Superblock

➤ Table - The fields of the Ext2 superblock

<u>Type</u>	<u>Field</u>	<u>Description</u>
__le32	s_inodes_count	Total number of inodes
__le32	s_blocks_count	Filesystem size in blocks
__le32	s_r_blocks_count	Number of reserved blocks
__le32	s_free_blocks_count	Free blocks counter
__le32	s_free_inodes_count	Free inodes counter
__le32	s_first_data_block	Number of first useful block (always 1)
__le32	s_log_block_size	Block size
__le32	s_log_frag_size	Fragment size
__le32	s_blocks_per_group	Number of blocks per group
__le32	s_frags_per_group	Number of fragments per group
__le32	s_inodes_per_group	Number of inodes per group
__le32	s_mtime	Time of last mount operation
__le32	s_wtime	Time of last write operation
__le16	s_mnt_count	Mount operations counter
__le16	s_max_mnt_count	Number of mount operations before check
__le16	s_magic	Magic signature

Ext2 FILE SYSTEM

❑ Superblock

➤ The fields of the Ext2 superblock

Type	Field	Description
__le16	s_state	Status flag
__le16	s_errors	Behavior when detecting errors
__le16	s_minor_rev_level	Minor revision level
__le32	s_lastcheck	Time of last check
__le32	s_checkinterval	Time between checks
__le32	s_creator_os	OS where filesystem was created
__le32	s_rev_level	Revision level of the filesystem
__le16	s_def_resuid	Default UID for reserved blocks
__le16	s_def_resgid	Default user group ID for reserved blocks
__le32	s_first_ino	Number of first nonreserved inode
__le16	s_inode_size	Size of on-disk inode structure
__le16	s_block_group_nr	Block group number of this superblock
__le32	s_feature_compat	Compatible features bitmap
__le32	s_feature_incompat	Incompatible features bitmap
__le32	s_feature_ro_compat	Read-only compatible features bitmap
__u8 [16]	s_uuid	128-bit filesystem identifier

Ext2 FILE SYSTEM

❑ Superblock

➤ The fields of the Ext2 superblock

Type	Field	Description
char [16]	s_volume_name	Volume name
char [64]	s_last_mounted	Pathname of last mount point
__le32	s_algorithm_usage_bitmap	Used for compression
__u8	s_prealloc_blocks	Number of blocks to preallocate
__u8	s_prealloc_dir_blocks	Number of blocks to preallocate for directories
__u16	s_padding1	Alignment to word
__u32 [204]	s_reserved	Nulls to pad out 1,024 bytes

- The s_inodes_count field stores the number of inodes, while the s_blocks_count field stores the number of blocks in the Ext2 filesystem.
- The s_log_block_size field expresses the block size as a power of 2, using 1,024 bytes as the unit.
- Thus, 0 denotes 1,024-byte blocks, 1 denotes 2,048-byte blocks, and so on.

Ext2 FILE SYSTEM

❑ Group Descriptor and Bitmap

- Each block group has its own group descriptor, an `ext2_group_desc` structure whose fields are illustrated in Table.

Table - The fields of the Ext2 group descriptor

Type	Field	Description
__le32	<code>bg_block_bitmap</code>	Block number of block bitmap
__le32	<code>bg_inode_bitmap</code>	Block number of inode bitmap
__le32	<code>bg_inode_table</code>	Block number of first inode table block
__le16	<code>bg_free_blocks_count</code>	Number of free blocks in the group
__le16	<code>bg_free_inodes_count</code>	Number of free inodes in the group
__le16	<code>bg_used_dirs_count</code>	Number of directories in the group
__le16	<code>bg_pad</code>	Alignment to word
__le32	<code>bg_reserved</code>	Nulls to pad out 24 bytes

- The `bg_free_blocks_count`, `bg_free_inodes_count`, and `bg_used_dirs_count` fields are used when allocating new inodes and data blocks.
- These fields determine the most suitable block in which to allocate each data structure.
- The bitmaps are sequences of bits, where the value 0 specifies that the corresponding inode or data block is free and the value 1 specifies that it is used.
- Because each bitmap must be stored inside a single block and because the block size can be 1,024, 2,048, or 4,096 bytes, a single bitmap describes the state of 8,192, 16,384, or 32,768 blocks.

Ext2 FILE SYSTEM

❑ **dumpe2fs** command

dumpe2fs - dump ext2/ext3/ext4 filesystem information

Description

- ★ **dumpe2fs** prints the super block and blocks group information for the filesystem present on *device*.

Options

- h only display the superblock information and not any of the block group descriptor detail information.

\$dumpe2fs -h /dev/sda1

- f force dumpe2fs to display a filesystem – both superblock and all of the block group descriptor detail information.

\$dumpe2fs -f /dev/sda1

- x print the detailed group information block numbers in hexadecimal format

\$dumpe2fs -x /dev/sda1

- V print the version number of dumpe2fs and exit.

- b print the blocks which are reserved as bad in the filesystem.

\$dumpe2fs -b /dev/sda1

Ext4 FILE SYSTEM

❏ Superblock

Terminal

↑ En 🔊 1:45 AM ⚙

```
ubuntu@ubuntu-vbox: ~  
$ sudo dumpe2fs -h /dev/sda1  
  
ubuntu@ubuntu-vbox:~$ sudo dumpe2fs -h /dev/sda1  
[sudo] password for ubuntu:  
dumpe2fs 1.42.13 (17-May-2015)  
Filesystem volume name: <none>  
Last mounted on: /  
Filesystem UUID: 29f11c93-b68b-4f55-b484-956b8a811053  
Filesystem magic number: 0xEF53  
Filesystem revision #: 1 (dynamic)  
Filesystem features: has_journal ext_attr resize_inode dir_index filetype needs_recovery extent flex_bg sparse_super large_file  
huge_file uninit_bg dir_nlink extra_isize  
Filesystem flags: signed_directory_hash  
Default mount options: user_xattr acl  
Filesystem state: clean  
Errors behavior: Continue  
Filesystem OS type: Linux  
Inode count: 3145728  
Block count: 12582656  
Reserved block count: 629132  
Free blocks: 9785916  
Free inodes: 2231087  
First block: 0  
Block size: 4096  
Fragment size: 4096  
Reserved GDT blocks: 1021  
Blocks per group: 32768  
Fragments per group: 32768  
Inodes per group: 8192  
Inode blocks per group: 512  
Flex block group size: 16  
Filesystem created: Fri Jan 12 17:22:11 2018  
Last mount time: Tue Dec 1 01:12:29 2020  
Last write time: Tue Dec 1 01:12:22 2020  
Mount count: 41
```

Ext4 FILE SYSTEM

❏ Superblock

```
Terminal
$sudo dumpe2fs -h /dev/sda1

Block size: 4096
Fragment size: 4096
Reserved GDT blocks: 1021
Blocks per group: 32768
Fragments per group: 32768
Inodes per group: 8192
Inode blocks per group: 512
Flex block group size: 16
Filesystem created: Fri Jan 12 17:22:11 2018
Last mount time: Tue Dec 1 01:12:29 2020
Last write time: Tue Dec 1 01:12:22 2020
Mount count: 41
Maximum mount count: -1
Last checked: Fri Jan 12 17:22:11 2018
Check interval: 0 (<none>)
Lifetime writes: 254 GB
Reserved blocks uid: 0 (user root)
Reserved blocks gid: 0 (group root)
First inode: 11
Inode size: 256
Required extra isize: 28
Desired extra isize: 28
Journal inode: 8
First orphan inode: 526729
Default directory hash: half_md4
Directory Hash Seed: 28c2268f-1c53-48f6-a2a8-bb3ac2d7287d
Journal backup: inode blocks
Journal features: journal_incompat_revoke
Journal size: 128M
Journal length: 32768
Journal sequence: 0x00115e97
Journal start: 10673

ubuntu@ubuntu-vbox:~$
```

Ext4 FILE SYSTEM

❑ Superblock & Block Group Descriptors

Terminal

↑↓ En 🔊 7:48 AM ⚙

\$sudo dumpe2fs -f /dev/sda1

```
dumpe2fs 1.42.13 (17-May-2015)
ubuntu@ubuntu-vbox:~$ sudo dumpe2fs -f /dev/sda1
dumpe2fs 1.42.13 (17-May-2015)
Filesystem volume name:   <none>
Last mounted on:         /
Filesystem UUID:          29f11c93-b68b-4f55-b484-956b8a811053
Filesystem magic number:  0xEF53
Filesystem revision #:    1 (dynamic)
Filesystem features:      has_journal ext_attr resize_inode dir_index filetype needs_recovery extent flex_bg sparse_super large_file huge_file uninit_bg dir_nlink extra_isize
Filesystem flags:         signed_directory_hash
Default mount options:    user_xattr acl
Filesystem state:         clean
Errors behavior:          Continue
Filesystem OS type:       Linux
Inode count:              3145728
Block count:              12582656
Reserved block count:     629132
Free blocks:              9785916
Free inodes:              2231087
First block:              0
Block size:               4096
Fragment size:            4096
Reserved GDT blocks:      1021
Blocks per group:         32768
Fragments per group:      32768
Inodes per group:         8192
Inode blocks per group:   512
Flex block group size:    16
Filesystem created:       Fri Jan 12 17:22:11 2018
Last mount time:          Tue Dec 1 01:12:29 2020
Last write time:          Tue Dec 1 01:12:22 2020
Mount count:              41
Maximum mount count:      -1
```

Ext4 FILE SYSTEM

❏ Superblock & Block Group Descriptors

```
Terminal
ubuntu@ubuntu-vbox: ~
$ sudo dumpe2fs -f /dev/sda1

Group 0: (Blocks 0-32767) [ITABLE_ZEROED]
Checksum 0xc223, unused inodes 0
Primary superblock at 0, Group descriptors at 1-3
Reserved GDT blocks at 4-1024
Block bitmap at 1025 (+1025), Inode bitmap at 1041 (+1041)
Inode table at 1057-1568 (+1057)
21444 free blocks, 102 free inodes, 1062 directories
Free blocks: 10229-10241, 10257-10261, 11342-32767
Free inodes: 15, 7465-7533, 7562, 7808-7838
Group 1: (Blocks 32768-65535) [ITABLE_ZEROED]
Checksum 0xb2cc, unused inodes 0
Backup superblock at 32768, Group descriptors at 32769-32771
Reserved GDT blocks at 32772-33792
Block bitmap at 1026 (bg #0 + 1026), Inode bitmap at 1042 (bg #0 + 1042)
Inode table at 1569-2080 (bg #0 + 1569)
2381 free blocks, 0 free inodes, 672 directories
Free blocks: 33814-33855, 33888-33999, 34008, 34016-34047, 34064-34067, 34116-34117, 34176-34231, 34240-34271, 34278-34292, 34511-35071, 35152-35157, 35164-35167, 35189-35192, 35194-35195, 35236-35237, 35262-35263, 35347, 35352-35388, 35444-35455, 35520-35551, 35568-35579, 35582-35647, 35712-35839, 36352-36735, 36768-36796, 36798-36799, 40960-41087, 41136-41151, 41182-41183, 41216-41471, 45056-45183, 45252-45265, 45311-45567
Free inodes:
Group 2: (Blocks 65536-98303) [ITABLE_ZEROED]
Checksum 0xde3f, unused inodes 6636
Block bitmap at 1027 (bg #0 + 1027), Inode bitmap at 1043 (bg #0 + 1043)
Inode table at 2081-2592 (bg #0 + 2081)
16824 free blocks, 6637 free inodes, 268 directories, 6636 unused inodes
Free blocks: 65576-65593, 65600-65663, 65731-65737, 65746-65752, 65761-65767, 65776-65782, 65791-65797, 65904-66047, 66136-66143, 66171-66303, 66307-66335, 66363-66559, 66689-66719, 66747-66879, 66907-66923, 66956-67023, 67040-67263, 67315-67327, 67393-67455, 67500-67583, 67649-67743, 67764-67839, 68561-69119, 71076-73599, 73707-73727, 86016-98303
Free inodes: 17940-24576
Group 3: (Blocks 98304-131071) [INODE_UNINIT, ITABLE_ZEROED]
Checksum 0x0168, unused inodes 8192
Backup superblock at 98304, Group descriptors at 98305-98307
```


Ext4 FILE SYSTEM

❏ Superblock & Block Group Descriptors

Terminal ↑↓ En 🔊 11:07 AM ⚙️

ubuntu@ubuntu-vbox: ~ **\$sudo dumpe2fs -x /dev/sda1**

```
Inode table at 12066336-12066847 (bg #368 + 7712)
32512 free blocks, 8192 free inodes, 0 directories, 8192 unused inodes
Free blocks: 12550144-12582655
Free inodes: 3137537-3145728
ubuntu@ubuntu-vbox:~$ sudo dumpe2fs -x /dev/sda1
dumpe2fs 1.42.13 (17-May-2015)
Filesystem volume name:   <none>
Last mounted on:         /
Filesystem UUID:          29f11c93-b68b-4f55-b484-956b8a811053
Filesystem magic number:  0xEF53
Filesystem revision #:    1 (dynamic)
Filesystem features:      has_journal ext_attr resize_inode dir_index filetype needs_recovery extent flex_bg sparse_super large_file
                           huge_file uninit_bg dir_nlink extra_isize
Filesystem flags:         signed_directory_hash
Default mount options:    user_xattr acl
Filesystem state:         clean
Errors behavior:          Continue
Filesystem OS type:       Linux
Inode count:               3145728
Block count:               12582656
Reserved block count:     629132
Free blocks:               9785916
Free inodes:               2231087
First block:               0
Block size:                4096
Fragment size:             4096
Reserved GDT blocks:      1021
Blocks per group:          32768
Fragments per group:       32768
Inodes per group:          8192
Inode blocks per group:    512
Flex block group size:     16
Filesystem created:        Fri Jan 12 17:22:11 2018
Last mount time:           Tue Dec  1 01:12:29 2020
```


Ext4 FILE SYSTEM

❏ Superblock & Block Group Descriptors

```
Terminal
ubuntu@ubuntu-vbox: ~$ sudo dumpe2fs -x /dev/sda1

Group 0: (Blocks 0x0000-0x7fff) [ITABLE_ZEROED]
Checksum 0xc223, unused inodes 0
Primary superblock at 0x0000, Group descriptors at 0x0001-0x0003
Reserved GDT blocks at 0x0004-0x0400
Block bitmap at 0x0401 (+1025), Inode bitmap at 0x0411 (+1041)
Inode table at 0x0421-0x0620 (+1057)
21444 free blocks, 102 free inodes, 1062 directories
Free blocks: 0x27f5-0x2801, 0x2811-0x2815, 0x2c4e-0x7fff
Free inodes: 0x000f, 0x1d29-0x1d6d, 0x1d8a, 0x1e80-0x1e9e
Group 1: (Blocks 0x8000-0xffff) [ITABLE_ZEROED]
Checksum 0xb2cc, unused inodes 0
Backup superblock at 0x8000, Group descriptors at 0x8001-0x8003
Reserved GDT blocks at 0x8004-0x8400
Block bitmap at 0x0402 (bg #0 + 1026), Inode bitmap at 0x0412 (bg #0 + 1042)
Inode table at 0x0621-0x0820 (bg #0 + 1569)
2381 free blocks, 0 free inodes, 672 directories
Free blocks: 0x8416-0x843f, 0x8460-0x84cf, 0x84d8, 0x84e0-0x84ff, 0x8510-0x8513, 0x8544-0x8545, 0x8580-0x85b7, 0x85c0-0x85df, 0x85e6-0x85f4, 0x86cf-0x88ff, 0x8950-0x8955, 0x895c-0x895f, 0x8975-0x8978, 0x897a-0x897b, 0x89a4-0x89a5, 0x89be-0x89bf, 0x8a13, 0x8a18-0x8a3c, 0x8a74-0x8a7f, 0x8ac0-0x8adf, 0x8af0-0x8afb, 0x8afe-0x8b3f, 0x8b80-0x8bff, 0x8e00-0x8f7f, 0x8fa0-0x8fbc, 0x8fbe-0x8fbf, 0xa000-0xa07f, 0xa0b0-0xa0bf, 0xa0de-0xa0df, 0xa100-0xa1ff, 0xb000-0xb07f, 0xb0c4-0xb0d1, 0xb0ff-0xb1ff
Free inodes:
Group 2: (Blocks 0x10000-0x17fff) [ITABLE_ZEROED]
Checksum 0x35b8, unused inodes 6636
Block bitmap at 0x0403 (bg #0 + 1027), Inode bitmap at 0x0413 (bg #0 + 1043)
Inode table at 0x0821-0x0a20 (bg #0 + 2081)
16905 free blocks, 6637 free inodes, 268 directories, 6636 unused inodes
Free blocks: 0x10028-0x10039, 0x10040-0x1007f, 0x100c3-0x100c9, 0x100d2-0x100d8, 0x100e1-0x100e7, 0x100f0-0x100f6, 0x100ff-0x10105, 0x10170-0x101ff, 0x10258-0x102ff, 0x10303-0x103ff, 0x10481-0x1053f, 0x1055b-0x1056b, 0x1058c-0x105cf, 0x105e0-0x106bf, 0x106f3-0x106ff, 0x10741-0x1077f, 0x107ac-0x107ff, 0x10841-0x1089f, 0x108b4-0x108ff, 0x10bd1-0x10dff, 0x115a4-0x11f7f, 0x11feb-0x11fff, 0x15000-0x17fff
Free inodes: 0x4614-0x6000
Group 3: (Blocks 0x18000-0x1ffff) [INODE_UNINIT, ITABLE_ZEROED]
```

SOC 3010

OPERATING SYSTEM

BLOCK GROUP DATA STRUCTURES

INODE & INODE TABLE

Ext2 FILE SYSTEM

☐ Inode

- The inode is a data structure in a Unix-style file system which describes a filesystem object such as a file or a directory.
- Each inode stores the attributes and disk block location(s) of the object's data.
- Filesystem object attributes may include metadata (times of last change, access, modification), as well as owner and permission data
- A file's inode number can be found using the **ls -li command**.
- The ls -li command prints the i-node number in the first column of the report.

Ext2/4 FILE SYSTEM

Ethereum [Running] - Oracle VM VirtualBox

File Machine View Input Devices Help

Terminal

ubuntu@ubuntu-vbox: ~

inode numbers associated with files & directories ls -li

```
ubuntu@ubuntu-vbox:~$ ls -li
530637 .
524289 ..
673112 Auction
531238 Ballot1
526695 .bash_history
530640 .bash_logout
530639 .bashrc
526991 build
524301 .cache
655368 .compiz
524579 .config
528389 contracts
536113 CourseraDocs
531233 .dbus
524557 Desktop
524315 .dmrc
524571 Documents
524558 Downloads
526987 EthereumWebInterface
530638 examples.desktop
2228245 ganache-1.1.0-beta.0-x86_64.AppImage
527886 .gconf
531000 .gnome
524836 .gnupg
1311944 go-ipfs
540035 go-ipfs_v0.6.0_linux-amd64.tar.gz
527882 .ICEauthority
1322198 .ipfs
524698 .local
537411 migrations
532373 .mozilla
524572 Music
535984 .node-gyp
534915 .node_repl_history
534973 .npm
535691 .npm-global
535692 .npmrc
524577 Pictures
530670 .pki
530641 .profile
524561 Public
524300 .sudo_as_admin_successful
524559 Templates
538693 test
538703 truffle-config.js
538704 truffle.js
525079 .vboxclient-clipboard.pid
530549 .vboxclient-display.pid
530656 .vboxclient-draganddrop.pid
530554 .vboxclient-seamless.pid
524578 Videos
524385 .Xauthority
524728 .xsession-errors
524302 .xsession-errors.old

ubuntu@ubuntu-vbox:~$ ls -li
total 108020
530637 drwxr-xr-x 33 ubuntu ubuntu 4096 Dec 1 01:14 .
524289 drwxr-xr-x 3 root root 4096 Jan 12 2018 ..
673112 drwxrwxr-x 6 ubuntu ubuntu 4096 Aug 14 07:53 Auction
531238 drwxrwxr-x 2 ubuntu ubuntu 4096 Nov 19 22:12 Ballot1
526695 -rw-r----- 1 ubuntu ubuntu 4007 Nov 25 08:15 .bash_history
530640 -rw-r--r-- 1 ubuntu ubuntu 220 Jan 12 2018 .bash_logout
530639 -rw-r--r-- 1 ubuntu ubuntu 3771 Jan 12 2018 .bashrc
526991 drwxrwxr-x 3 ubuntu ubuntu 4096 Aug 14 05:23 build
524301 drwx----- 19 ubuntu ubuntu 4096 Nov 3 06:44 .cache
655368 drwx----- 3 ubuntu ubuntu 4096 Jan 12 2018 .compiz
524579 drwx----- 21 ubuntu ubuntu 4096 Aug 16 12:35 .config
528389 drwxr-xr-x 2 ubuntu ubuntu 4096 Aug 14 07:49 contracts
536113 drwxrwxr-x 3 ubuntu ubuntu 4096 May 12 2018 CourseraDocs
```

Address Desktop 5:31 PM 12/3/2020

□ Inode Table

- The inode table consists of a series of consecutive blocks, each of which contains a predefined number of inodes.
- The block number of the first block of the inode table is stored in the **bg_inode_table** field of the group descriptor.
- All inodes have the same size: 128 bytes (ext2).
- A 1,024-byte block contains 8 inodes, while a 4,096-byte block contains 32 inodes.
- All inodes have the same size: 256 bytes (ext4).
- A 1,024-byte block contains 4 inodes, while a 4,096-byte block contains 16 inodes.
- To figure out how many blocks are occupied by the inode table, divide the total number of inodes in a group (stored in the **s_inodes_per_group** field of the superblock) by the number of inodes per block.

□ Inode Table

Q) Given a Ext2 file system having 8192 inodes in each blockgroup with blocksize 4KB, determine the total number of blocks required by the Inode Table

- In Ext2 filesystem, Inode size = 128 bytes
- Given Block size = 4KB,
Number inodes per Blockgroup = 8192
- A 4KB block can contain 32 inodes
- **Hence, total number of blocks in the Inode table**
= Number of inodes per blockgroup / Number of inodes per block
= $8192 / 32 = 256$ blocks

Q) Given a Ext4 file system having 8192 inodes in each blockgroup with blocksize 4KB, determine the total number of blocks required by the Inode Table

- In Ext4 filesystem, Inode size = 256 bytes
- Given Block size = 4KB,
Number inodes per Blockgroup = 8192
- A 4KB block can contain 16 inodes
- **Hence, total number of blocks in the Inode table**
= Number of inodes per blockgroup / Number of inodes per block
= $8192 / 16 = 512$ blocks

SOC 3010

OPERATING SYSTEM

BLOCK GROUP DATA STRUCTURES

INODE STRUCTURE

Ext2 FILE SYSTEM

□ Each Ext2 disk inode is an `ext2_inode` structure whose fields are in Table below

Type	Field	Description
__le16	i_mode	File type and access rights
__le16	i_uid	Owner identifier
__le32	i_size	File length in bytes
__le32	i_atime	Time of last file access
__le32	i_ctime	Time that inode last changed
__le32	i_mtime	Time that file contents last changed
__le32	i_dtime	Time of file deletion
__le16	i_gid	User group identifier
__le16	i_links_count	Hard links counter
__le32	i_blocks	Number of data blocks of the file
__le32	i_flags	File flags
Union	osd1	Specific operating system information
__le32 [EXT2_N_BLOCKS]	i_block	Pointers to data blocks
__le32	i_generation	File version (used when the file is accessed by a network filesystem)
__le32	i_file_acl	File access control list
__le32	i_dir_acl	Directory access control list
__le32	i_faddr	Fragment address
Union	osd2	Specific operating system information

15

Ext2 FILE SYSTEM

□ Inode - i_size, i_blocks & i_block[15]

- In particular, the **i_size** field stores the effective length of the file in bytes, while the **i_blocks** field stores the number of data blocks (in units of 512 bytes) that have been allocated to the file.
- For example: If a file contains 8KB data, **i_size** field contains 8192 bytes indicating the size of the file whereas **i_blocks** field contains 16 indicating the number of sectors (sector size=512 bytes) allocated to store 8KB (i.e., 2 blocks if block size=4KB)
- The values of **i_size** and **i_blocks** are not necessarily related.
- Because a file is always stored in an integer number of blocks, a nonempty file receives at least one data block and **i_size** may be smaller than $512 \times i_blocks$.
- On the other hand, a file may contain holes. In that case, **i_size** may be greater than $512 \times i_blocks$.
- The **i_block** field is an array of **EXT2_N_BLOCKS** (usually 15) pointers to blocks used to identify the data blocks allocated to the file

Ext2 FILE SYSTEM

❑ Inode – i_dir_acl & i_size (64-bit file size)

- The 32 bits reserved for the i_size field limit the file size to 4 GB.
- Actually, the highest-order bit of the i_size field is not used, so the maximum file size is limited to 2 GB.
- However, the Ext2 filesystem includes a "dirty trick" that allows larger files on systems that sport a 64-bit processor such as AMD's Opteron or IBM's PowerPC G5.
- Essentially, the i_dir_acl field of the inode, which is not used for regular files, represents a 32-bit extension of the i_size field.
- Therefore, the file size is stored in the inode as a 64-bit integer.
- The 64-bit version of the Ext2 filesystem is somewhat compatible with the 32-bit version because an Ext2 filesystem created on a 64-bit architecture may be mounted on a 32-bit architecture, and vice versa.
- On a 32-bit architecture, a large file cannot be accessed, unless opening the file with the O_LARGEFILE flag set



❑ Inode Table

- VFS model requires each file to have a different inode number.
- In Ext2, there is no need to store on disk a mapping between an inode number and the corresponding block number because the latter value can be derived from the block group number and the relative position inside the inode table.
- For example, suppose that each block group contains 4,096 inodes and that we want to know the address on disk of inode 13,021.
- In Blockgroup 0 – inode 0 to 4095,
- In Blockgroup 1 – inode 4096 to 8191
- In Blockgroup 2 – inode 8192 to 12287
- In Blockgroup 3 – inode 12288 to 16383
- In this case, the inode belongs to the Block group 3 and its disk address is stored in the $(13021 - 12288 = 733)$ 733rd entry of the corresponding inode table.
- The inode number is just a key used by the Ext2 routines to retrieve the proper inode descriptor on disk quickly.

LINUX DISK FILE SYSTEM

Q) Suppose that each block group contains 4,096 inodes and that we want to know the address of inode 13,021 on disk .
That is, Compute Blockgroup number and which entry of the corresponding Node table contain this inode 13,021.

In Blockgroup 0 – inode 0 to 4095,
In Blockgroup 1 – inode 4096 to 8191
In Blockgroup 2 – inode 8192 to 12287
In Blockgroup 3 – inode 12288 to 16383

In this case, the inode belongs to the Blockgroup number = $13021/4096=3$, i.e., Block group 3 and its disk address is stored in the $(13021- 3 \times 4096 = 13021 - 12288 = 733)$ 733rd entry of the corresponding inode table.

Extended Attributes of an Inode

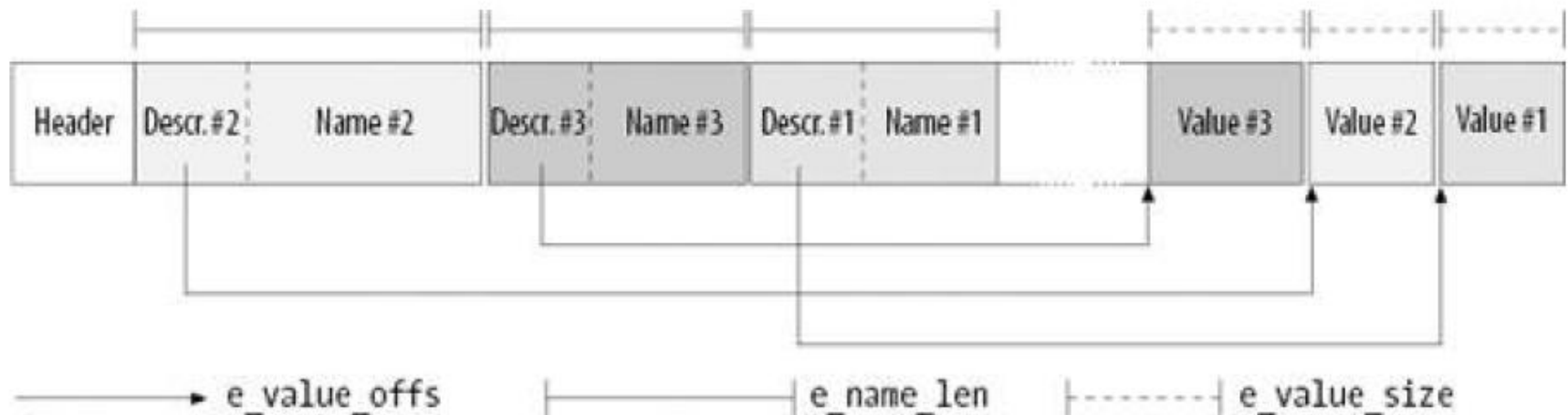
- The Ext2 inode format is a kind of straitjacket for filesystem designers.
- The length of an inode must be a power of 2 to avoid internal fragmentation in the blocks that store the inode table.
- Actually, most of the 128 characters of an Ext2 inode are currently packed with information, and there is little room left for additional fields.
- On the other hand, expanding the inode length to 256 would be quite wasteful, besides introducing compatibility problems between Ext2 filesystems that use different inode lengths.
- Extended attributes have been introduced to overcome the above limitation.
- These attributes are stored on a disk block allocated outside of any inode.

Extended Attributes of an Inode

- The `i_file_acl` field of an inode points to the block containing the extended attributes.
- Different inodes that have the same set of extended attributes may share the same block.
- Each extended attribute has a name and a value.
- Both of them are encoded as variable length arrays of characters, as specified by the `ext2_xattr_entry` descriptor.

Extended Attributes of an Inode

- Figure shows the layout in Ext2 of the extended attributes inside a block.
- Each attribute is split in two parts: the ext2_xattr_entry descriptor together with the name of the attribute are placed at the beginning of the block, while the value of the attribute is placed at the end of the block.
- The entries at the beginning of the block are ordered according to the attribute names, while the positions of the values are fixed, because they are determined by the allocation order of the attributes.



Extended Attributes of an Inode

- There are many system calls used to set, retrieve, list, and remove the extended attributes of a file.
- The **setxattr()** , **lsetxattr()** , and **fsetxattr()** system calls set an extended attribute of a file; essentially, they differ in how symbolic links are handled, and in how the file is specified (either passing a pathname or a file descriptor).
- Similarly, the **getxattr()** , **lgetxattr()** , and **fgetxattr()** system calls return the value of an extended attribute.
- The **listxattr()** , **lstatxattr()** , and **flistxattr()** list all extended attributes of a file.
- Finally, the **removexattr()** , **lremovexattr()** , and **fremovexattr()** system calls remove an extended attribute from a file.

Access Control Lists



- Access control lists were proposed a long time ago to improve the file protection mechanism in Unix filesystems.
- Instead of classifying the users of a file under three classes - owner, group, and others, an access control list (ACL) can be associated with each file.
- A user may specify for each of his files the names of specific users (or groups of users) and the privileges to be given to these users.
- Linux fully supports ACLs by making use of inode extended attributes.
- As a matter of fact, extended attributes have been introduced mainly to support ACLs.
- Therefore, the `chacl( )`, `setfacl( )`, and `getfacl( )` library functions, which allow you to manipulate the ACLs of a file, rely essentially upon the `setxattr( )` and `getxattr( )` system calls.
- ACLs are supported nowadays on different filesystem types on many UNIX-like systems

Access Control Lists

❑ Setfacl command

- * Setfacl utility sets Access Control Lists (ACLs) of files and directories.
- * On the command line, a sequence of commands is followed by a sequence of files (which in turn can be followed by another sequence of commands, ...).
- * The options **-m**, and **-x** expect an ACL on the command line.
- * Multiple ACL entries are separated by comma characters (',').
- * The options **-M**, and **-X** read an ACL from a file or from standard input.
- * The **--set** and **--set-file** options set the ACL of a file or a directory. The previous ACL is replaced. ACL entries for this operation must include permissions.
- * The **-m** (**--modify**) and **-M** (**--modify-file**) options modify the ACL of a file or directory. ACL entries for this operation must include permissions.
- * The **-x** (**--remove**) and **-X** (**--remove-file**) options remove ACL entries.
- * When reading from files using the **-M**, and **-X** options, setfacl accepts the output getfacl produces. There is at most one ACL entry per line.

❑ Examples

- * Granting an additional user read access
\$setfacl -m u:aziz:r file
- * Revoking write access from all groups and all named users (using the effective rights mask)
\$setfacl -m m::rx file
- * Removing a named group entry from a file's ACL
\$setfacl -x g:staff file
- * Copying the ACL of one file to another
\$getfacl file1 | setfacl --set-file=- file2
- * Copying the access ACL into the Default ACL
\$getfacl --access dir | setfacl -d -M- dir

❑ getfacl command

- * For each file, getfacl displays the file name, owner, the group, and the Access Control List (ACL). If a directory has a default ACL, getfacl also displays the default ACL

Access Control Lists

```
Terminal
ubuntu@ubuntu-vbox: ~

ubuntu@ubuntu-vbox:~/CourseraDocs$ ls
2_deploy_contracts.js  app.js  Auction.zip  Ballot2.zip  Ballot.sol  Coin.zip  test.js  truffle.js
ubuntu@ubuntu-vbox:~/CourseraDocs$ getfacl Ballot.sol
# file: Ballot.sol
# owner: ubuntu
# group: ubuntu
user::rwx
group::r-x
other::r-x

ubuntu@ubuntu-vbox:~/CourseraDocs$ getfacl test.js
# file: test.js
# owner: ubuntu
# group: ubuntu
user::rwx
group::r-x
other::r-x

ubuntu@ubuntu-vbox:~/CourseraDocs$ cd
ubuntu@ubuntu-vbox:~$ ls
Auction      CourseraDocs  EthereumWebInterface  go-ipfs_v0.6.0_linux-amd64.tar.gz  Public      truffle.js
Ballot1      Desktop      examples.desktop      migrations      Templates    Videos
build        Documents    ganache-1.1.0-beta.0-x86_64.AppImage  Music          test
contracts    Downloads    go-ipfs               Pictures        truffle-config.js

ubuntu@ubuntu-vbox:~$ getfacl go-ipfs
# file: go-ipfs
# owner: ubuntu
# group: ubuntu
user::rwx
group::rwx
other::r-x

ubuntu@ubuntu-vbox:~$
```

DISK FILE SYSTEM

□ Question

- * In a Ext2 file system, if disk block size = 1KB, # of inodes = 2048 inodes and 4K Blockgroups then determine the following**
 - **a) Size of superblock = block**
 - **b) Number of blockgroup descriptors =**
 - **c) Size of Blockgroup descriptors = blocks**
 - **d) Size of data block bitmap = block**
 - **e) Size of inode bitmap = block**
 - **f) Size of inode = bytes**
 - **g) Number of inodes =**
 - **h) Size of inode table = bytes**
= blocks
 - **i) Total number of data blocks in the Block group =**
 - **j) Number of data blocks available for storing file data =**
 - **k) Size of one blockgroup =**
 - **l) Size of the disk partition =**

DISK FILE SYSTEM

□ Question

- * In a Ext2 file system, if disk block size = 1KB, # of inodes = 2048 inodes and 4K Blockgroups then determine the following**
 - **a) Size of superblock = 1 block**
 - **b) Number of Blockgroup descriptors = 4K**
 - **c) Size of blockgroup descriptors = 4K * 24 = 96 Blocks**
where size of 1 Block Group Descriptor = 24 bytes
 - **d) Size of data block bitmap = 1 block**
 - **e) Size of inode bitmap = 1 block**
 - **f) Size of inode = 128 bytes**
 - **g) Number of inodes = 2048**
 - **h) Size of inode table = size of inode x number of inodes = 128 x 2048 = 256KB = 256 blocks**
 - **i) Total number of data blocks in the Block group = Blocksize x 8 = 1KB x 8 = 1024x8 = 8192 KB = 8192 blocks**
 - **j) Number of data blocks available for storing file data**
= Total no. of data blocks in Block group – (1block(super block) + 96 blocks(Blockgroup descriptors)+ 1 Block (data block bitmap) + 1block(inode bitmap) + 256blocks(Inode table)) = (8192 – 355) = 7837 blocks
 - **k) Size of one blockgroup = 8192 blocks x size of each block = 8Kx1K = 8MB**
 - **l) Size of the disk partition = No. of Blockgroups x size of one Blockgroup = 4K x 8MB = 32GB**

DISK FILE SYSTEM

□ Question

*** Given a 64GB partition having Ext2 file system with disk block size = 4KB, # of inodes = 8192 inodes then determine the following :**

- **a) Total number of blocks in a blockgroup =**
- **b) Size of one Blockgroup =**
- **c) Total number of Blockgroups =**
- **d) Size of superblock =**
- **e) Size of blockgroup descriptors =**
- **f) Size of data block bitmap =**
- **g) Size of inode bitmap =**
- **h) Size of inode =**
- **i) Number of inodes =**
- **j) Size of inode table =**
- **k) Number of data blocks available for storing file data =**

DISK FILE SYSTEM

□ Question

- * Given a 64GB partition having Ext2 file system with disk block size = 4KB, # of inodes = 8192 inodes then determine the following :
- a) Total number of blocks in a blockgroup = $\text{blocksize} \times 8 = 4096 \times 8 = 32768 \text{ blocks}$
 - b) Size of one Blockgroup = $32768 \times 4\text{KB} = 32\text{KB} \times 4\text{KB} = 128\text{MB}$
 - c) Total number of Blockgroups = $64\text{GB} / 128\text{MB} = 512$
 - d) Size of superblock = 1 block
 - e) Size of blockgroup descriptors = $512 \times 24 / 4096 = 3 \text{ blocks}$
 - f) Size of data block bitmap = 1 block
 - g) Size of inode bitmap = 1 block
 - h) Size of inode = 128 bytes
 - i) Number of inodes = 8192
 - j) Size of inode table = size of inode $\times$ number of inodes
= $128 \times 8192 = 1024\text{KB} = 1024\text{KB} / 4\text{KB} = 256 \text{ blocks}$
 - k) Number of data blocks available for storing file data
= Total no. of data blocks in Block group – (1block(super block) + 3 blocks(Blockgroup descriptors)+ 1 Block (data block bitmap) + 1block(inode bitmap) + 256blocks(Inode table))
= $32768 - 262 = 32506 \text{ blocks}$

DISK FILE SYSTEM

□ Questions

- * State the purpose of sparse_super feature flag in Ext4 file system.**
- * In a Ext2 file system, if disk block size = 1KB, # of inodes = 2048 inodes and 2K Blockgroups then determine the following**
 - a) size of superblock
 - b) size of blockgroup descriptors
 - c) size of data block bitmap
 - d) size of inode bitmap
 - e) size of inode
 - f) Number of inodes
 - g) size of inode table
 - h) Total number of data blocks in the Block group
 - i) Number of data blocks available for storing file data
 - g) Size of one blockgroup
 - h) Size of the disk partition
- * If disk block size = 2 KB, # of inodes = 8192 inodes and 16K Blockgroups then determine the following**
 - a) size of superblock,
 - b) size of blockgroup descriptors,
 - c) size of data block bitmap,
 - d) size of inode bitmap,
 - e) size of inode and size of inode table,
 - f) number of data blocks and size of data blocks
 - g) Size of one blockgroup
 - h) Size of the disk partition.
 - i) Number of data blocks available for storing file data
 - g) Size of one blockgroup
 - h) Size of the disk partition

DISK FILE SYSTEM

□ Questions

- * If disk block size = 8 KB, # of inodes = 8192 inodes and Partition size = 64GB then determine the following**
 - a) size of superblock,**
 - b) Number of blockgroup descriptors,**
 - c) Size of blockgroup descriptors,**
 - d) Size of data block bitmap,**
 - e) Size of inode bitmap,**
 - f) Size of inode and size of inode table,**
 - g) Number of data blocks**
 - h) Size of data block**
 - i) Number of Blocks in blockgroup**
 - j) Size of one blockgroup**
 - k) Number of blockgroups**
- * If disk block size = 16KB, # of inodes = 8192 inodes and Partition size = 128GB then determine the following**
 - a) size of superblock,**
 - b) Number of blockgroup descriptors,**
 - c) Size of blockgroup descriptors,**
 - d) Size of data block bitmap,**
 - e) Size of inode bitmap,**
 - f) Size of inode and size of inode table,**
 - g) Number of data blocks**
 - h) Size of data block**
 - i) Number of Blocks in blockgroup**
 - j) Size of one blockgroup**
 - k) Number of blockgroups**
- * How many block groups are there in a given 512 GB Ext2 partition with a 4-KB block size and 8192 inodes in each block group.**

DISK FILE SYSTEM

□ Questions

- * Explain what is meant by file extent.**
- * Write the various fields in a blockgroup descriptor data structure `ext2_group_desc` and describe the information provided by each of the field in the blockgroup descriptor data structure.**
- * Write the various fields in inode (file descriptor) data structure `ext2_inode` and describe the information provided by each of the field in the inode data structure.**
- * Explain the purpose of following fields in an inode data structure :**
 - a) `i_mode`,** **b) `i_uid`,** **c) `i_size`,**
 - d) `i_links_count`,** **e) `i_blocks`,** **f) `i_block[15]`,**
 - g) `i_file_acl`,** **h) `i_dir_acl`,** **i) `i_atime`,**
 - j) `i_ctime`,** **k) `i_mtime`,** **l) `i_dtime`, m) `i_gid`**
- * What is Access Control List in Ext2/Ex4 FileSystem?**
- * Write the linux command to create access control list for individual users or groups for a given file.**
- * Explain the following Unix commands by giving the correct syntax and taking some examples**
 - a) `setfacl`** **b) `getfacl`**
- * State the reason for including Extended attributes for Inode Objects?**
- * Explain how `i_file_acl` field of the inode is used to create extended attributes of an inode.**
- * Describe the layout of an Ex2 extended attributes structure inside a block with the help of a diagram**

DISK FILE SYSTEM

□ Questions

- * State the purpose of **sparse_super** feature flag in Ext4 file system.

- * If disk block size = 1KB, # of inodes = 8192 inodes and 8K Blockgroups then determine the following
 - a) size of superblock,
 - b) size of blockgroup descriptors,
 - c) size of data block bitmap,
 - d) size of inode bitmap,
 - e) size of inode and size of inode table,
 - f) number of data blocks and size of data blocks
 - g) Size of one blockgroup
 - h) Size of the disk partition.

- * If disk block size = 2 KB, # of inodes = 8192 inodes and 16K Blockgroups then determine the following
 - a) size of superblock,
 - b) size of blockgroup descriptors,
 - c) size of data block bitmap,
 - d) size of inode bitmap,
 - e) size of inode and size of inode table,
 - f) number of data blocks and size of data blocks
 - g) Size of one blockgroup
 - h) Size of the disk partition.

DISK FILE SYSTEM

Questions

- * Specify the File size upper limits for data block addressing in the following table :

Block Size	Direct	1-Indirect	2-indirect	3-indirect
1KB				
2KB				
4KB				
8KB				
16KB				

- * Explain what is meant by a file hole. Give an example.
- * Write the file size and total number of blocks assigned to a file **hole** when the following commands are executed. Also show the `i_block` array and file `/tmp/hole` data block contents.

```
$ echo -n "X" | dd of=/tmp/hole bs=1024 seek=6
$ echo -n "X" | dd of=/tmp/hole bs=4096 seek=6
$ echo "HELLO" | dd of=/tmp/hole bs=1024 seek=10
$ echo "HELLO" | dd of=/tmp/hole bs=4096 seek=10
```

- * State the purpose of the following commands :

```
$ mke2fs -t ext4 -b 4096 /dev/sda1
$fsck
```

- * State the purpose of the following commands :

```
$ mount -t ext4 /dev/fd0 /floppy
$mount -t ext4 -o data=writeback /dev/sda2 /jdisk
```

- * State the purpose of the following commands :

```
$setfacl -m "u:user_name:rwX" file1
$setfacl -m "g:group_name:rwX" file1
$getfacl file1
```

DISK FILE SYSTEM

❑ **Questions**

- * Write a linux command to display only the contents of the superblock of ext4 filesystem stored on the disk partition /dev/sda1.
- * Write a linux command to display only the contents of the block group descriptors of ext4 filesystem stored on the disk partition /dev/sda1.
- * State what happens when the following linux commands are executed. Clearly write the outputs obtained.
 - \$ dumpe2fs -h /dev/sda1**
 - \$ dumpe2fs -g /dev/sda1**
- * State what happens when the following linux commands are executed. Clearly write the outputs obtained.
 - \$ tune2fs -l /dev/sda1**
 - \$ tune2fs -l /dev/sda1 | grep "Default mount options"**
 - \$ tune2fs -o acl /dev/sda1**
- * State what happens when the following linux commands are executed. Clearly write the outputs obtained.
 - \$ lsblk -O**
 - \$ fdisk -l**
- * State what happens when the following linux commands are executed. Clearly write the outputs obtained.
 - \$ ls -la**
 - \$ lscpu**
 - \$ lsipc**
- * Define Journaling and explain why is it required.
- * Explain the idea behind Ext3/4 Journaling to perform each high-level change to the filesystem.

DISK FILE SYSTEM

□ Questions

- * Describe how journaling is used to recover from system crashes.**
- * Explain the two kinds of blocks in filesystem that need to be logged to the Journal.**
- * How does the kernel check whether the filesystem was unmounted properly previously or not?**
- * State the purpose of `s_mount_state` field of the superblock on disk.**
- * Explain the three different Journaling modes provided by Ext3/Ext4. State the merits and demerits of each of these modes.**
- * Explain the following Journaling modes indicating clearly their advantages and disadvantages.**
 - a) Journal**
 - b) ordered**
 - c) writeback**
- * Explain how Journaling mode of the Ext3/Ext4 filesystem can be specified at the time of mounting the filesystem.**
- * Write the linux command to mount an Ext3 filesystem stored in the `/dev/sda2` partition on the `/jdisk` mount point with the "writeback" mode.**
- * The Ext3 journal is usually stored in a hidden file namedlocated in the root directory of the filesystem.**

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 18 of Text Book

Text Book

➤ Understanding the Linux Kernel

- Daniel P/ Bovet and Marco Cesati
Edition, O' Reilly Media 2005, ISBN 978-0596005658**

SOC 3010
OPERATING SYSTEM

END OF
WEEK 11 LECTURE 2

FILE : ON DISK FILE
SYSTEM

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

